

SecureDrive — Literature Survey

Privacy-Preserving Cloud File Storage: A Comparative Review

Yashaswini L & Raghuveer Verma · IIT Dharwad, CSE · BTP 2026

1. Introduction

The dominant cloud storage paradigm — as employed by Google Drive, Dropbox, and Microsoft OneDrive — relies on server-side encryption: the service provider holds the cryptographic keys and can therefore decrypt user data. This means a server compromise, insider threat, or lawful access request can expose user files. Privacy-preserving cloud storage addresses this through client-side encryption: all cryptographic operations run on the user's device, and keys never leave the client. The server stores only ciphertext.

This survey examines three established privacy-preserving cloud storage solutions: Cryptomator, Proton Drive, and Filen. For each, we analyse the cryptographic architecture, key management scheme, threat surface, and limitations. The findings identify the research gap that SecureDrive's blockchain-anchored audit trail addresses.

2. Cryptomator

2.1 Overview

Cryptomator is a fully open-source, client-side encryption layer for existing cloud providers. It does not provide its own cloud backend — instead it creates an encrypted virtual drive (vault) that the user's existing cloud service (Google Drive, Dropbox, etc.) synchronises. Cryptomator is available as a desktop application for Windows, macOS, and Linux, and mobile apps for iOS and Android. A web-based variant called Cryptomator Hub adds enterprise key management using ECDH-ES.

2.2 Vault and Key Architecture

Each Cryptomator vault contains a vault configuration file (vault.cryptomator) — a signed JWT specifying the cipher combination (SIV_GCM or SIV_CTRMAC) and vault format version. The vault uses two separate 256-bit master keys: an encryption master key and a MAC master key. These are generated using a cryptographically secure pseudorandom number generator (CSPRNG).

Key derivation from password:

- The user's vault password is processed using scrypt with parameters: $N=32768$ (cost), $r=8$ (block size), $p=1$ (parallelism)
- scrypt produces a 256-bit KEK (Key Encryption Key)
- The KEK encrypts both master keys using AES Key Wrap (RFC 3394)
- The wrapped keys, scrypt salt, and parameters are stored in masterkey.cryptomator — a JSON file in the vault root
- The vault configuration JWT is signed using the concatenation of both master keys, preventing undetected tampering

File encryption:

- Files are encrypted using AES-256-GCM in SIV (Synthetic IV) mode — a nonce-misuse-resistant construction that protects against IV reuse
- File names are also encrypted to prevent metadata leakage via the directory structure
- Each file uses a unique derived key — the master key is never used directly for file encryption
- Files are split into 32 KiB chunks for processing efficiency

2.3 Limitations

- Requires desktop or mobile application installation — not web-native; no browser-based interface
- Does not provide cloud storage — entirely dependent on third-party providers for persistence
- No file sharing between users — the only sharing mechanism is sharing the vault password, which grants access to all vault contents
- No audit trail of any kind — no record of when files were accessed, by whom, or from where
- Access pattern leakage — the cloud provider observes file modification timestamps and sizes even though content is encrypted
- script cost parameter $N=32768$ is below the NIST 2023 recommended minimum for equivalent security in modern threat environments

3. Proton Drive

3.1 Overview

Proton Drive is an end-to-end encrypted cloud storage service developed by Proton AG (Switzerland). It provides a web interface, desktop applications, and mobile apps. Unlike Cryptomator, Proton Drive provides its own encrypted cloud backend. The security model is based on OpenPGP with a hierarchical, per-node key structure reflecting the folder tree.

3.2 Cryptographic Architecture

Data model:

Proton Drive organises content in a tree structure of nodes (files and folders) and links (parent-child relationships). Each node has an associated asymmetric key pair (node key) and a node passphrase. This hierarchical structure means decryption proceeds level by level from the share root down to the target file.

Key hierarchy:

- Each user has an address key (OpenPGP key pair) derived from their account credentials
- A share passphrase (32 random bytes) is generated per share; it is encrypted with each member's address key
- The share passphrase unlocks the share key, which in turn unlocks the root node key
- Each node (file or folder) has its own asymmetric key pair; the node passphrase is encrypted by the parent folder's node key
- File content is split into 4 MB blocks; each block is encrypted with the file's node key using AES-256 via OpenPGP
- File and folder names are encrypted with the parent folder's node key

File sharing:

- Sharing between Proton users is implemented by re-encrypting the share passphrase with the recipient's address key — the file content is unchanged
- Sharing with non-Proton users uses a URL-based mechanism: a share passphrase is encrypted with a password, which is embedded in the URL fragment (not sent to Proton's servers)

3.3 Limitations

- The server-side implementation is proprietary and closed source — users cannot independently verify Proton's claimed security properties
- The audit log is server-maintained by Proton — users must trust Proton's logs and have no cryptographic mechanism to verify access history independently

- OpenPGP, while well-established, is a legacy standard; modern constructions using purpose-built AEAD schemes (AES-GCM) are considered more efficient and easier to implement correctly
- The hierarchical key structure creates a large attack surface — a compromise of a high-level node key could expose all descendant nodes
- Account recovery mechanisms (which Proton supports) inherently weaken the key hierarchy by creating alternative decryption paths
- Technical documentation is limited to a blog post — no formal threat model, security proofs, or framework compliance documentation is published

4. Filen

4.1 Overview

Filen is a commercial end-to-end encrypted cloud storage service based in Germany and subject to strict European (GDPR) and German data protection law. It provides a web interface, desktop applications, and mobile apps. Filen uses AES-256-GCM for file encryption — the same primitive as SecureDrive. However, Filen is entirely closed source and publishes no formal technical documentation of its cryptographic architecture.

4.2 Known Cryptographic Details

Based on Filen's published security page, the following is known:

- File encryption: AES-256-GCM (confirmed)
- Filen claims quantum resistance — AES-256 is indeed considered quantum-resistant under Grover's algorithm, which reduces its effective security to 128 bits, still well within acceptable margins
- Servers are physically owned by Filen and located exclusively in ISO 27001-certified Tier III-IV data centres in Germany
- Key derivation algorithm, iteration counts, and key hierarchy details are not publicly documented

4.3 Limitations

- Entirely closed source — the cryptographic implementation cannot be independently audited or verified
- No technical whitepaper or formal security documentation is available — claims are limited to a marketing security page
- No audit trail of any kind — no record of file access events is provided to users
- As a commercial product, long-term availability depends on Filen's business continuity
- Key derivation parameters are undisclosed — it is impossible to assess PBKDF2 iteration count compliance against NIST recommendations
- No formal threat model, MITRE ATT&CK mapping, or security framework compliance is documented

5. Comparative Analysis

Feature	Cryptomator	Proton Drive	Filen
File Encryption	AES-256-GCM (SIV_GCM)	AES-256 via OpenPGP	AES-256-GCM
Key Derivation	scrypt (N=32768, r=8)	bcrypt + SRP + custom KDF	PBKDF2 (iteration count undisclosed)
Key Architecture	Two master keys (encryption + MAC) wrapped with KEK	Per-node asymmetric key tree (OpenPGP)	Password → master key → per-file keys

Encryption Location	Desktop/mobile app only	Browser + native apps	Browser + native apps
Open Source	☒ Fully open source	☐ Client open, server closed	☒ Closed source
Web Native	☒ App required	☒ Web + apps	☒ Web + apps
File Sharing	☒ Password sharing only	☒ Per-file key re-encryption	☒ Per-file key re-encryption
Verifiable Audit Trail	☒ None	☒ Server logs only (trust Proton)	☒ None
Server Breach Impact	Encrypted only	Encrypted only	Encrypted only
Blockchain Audit	☒ None	☒ None	☒ None
Technical Transparency	☒ Full docs published	☐ Blog post only	☒ Marketing page only

6. Research Gap: The Missing Audit Trail

The survey identifies a consistent and critical gap across all three examined solutions: the complete absence of a cryptographically verifiable, user-controlled audit trail.

In all three systems, either no audit logging exists at all (Cryptomator, Filen) or audit logs are maintained exclusively by the service provider on its own servers (Proton Drive). This creates a fundamental trust dependency:

- Users cannot independently verify whether their files were accessed without authorisation
- Users cannot prove that server-side logs have not been altered or deleted
- In the event of a server compromise, users have no cryptographic evidence of whether their specific files were accessed

This gap has practical consequences in security-critical domains. A healthcare professional storing patient records, a legal practitioner managing confidential documents, or a journalist protecting source files requires more than an assurance that the server was not breached — they require cryptographic proof of access history that cannot be forged or deleted by any single party, including the service provider itself.

SecureDrive addresses this gap through a blockchain-anchored audit trail. Every file access event generates a cryptographic hash that is anchored to the Ethereum Sepolia public blockchain. Because the blockchain is decentralised and immutable:

- No single party — including SecureDrive's own server — can alter or delete anchored access records
- Users can independently verify their complete access history by querying the blockchain directly, without trusting any intermediary
- A server compromise cannot retroactively conceal unauthorised access events that have already been anchored on-chain

7. SecureDrive — Novel Contribution and Positioning

Property	SecureDrive
File Encryption	AES-256-GCM with unique IV per file — same primitive as Cryptomator and Filen

Key Derivation	PBKDF2-SHA256 with 600,000 iterations — NIST SP 800-132 (2023) compliant. Exceeds Cryptomator's scrypt default and Filen's undisclosed iteration count
Key Architecture	Three-layer hierarchy: password-derived key → Master Key → per-file CEK. Enables password change without re-encrypting files; limits breach blast radius to one file per compromised CEK
Encryption Location	Fully browser-native using Web Crypto API — no installation required (advantage over Cryptomator)
Open Source	Fully open source — entire cryptographic stack auditable (matches Cryptomator, exceeds Proton Drive and Filen)
Technical Documentation	Formal STRIDE threat model, MITRE ATT&CK mapping, NIST SSDF compliance documentation — exceeds all three compared solutions
Verifiable Audit Trail	Blockchain-anchored tamper-evident audit trail on Ethereum Sepolia — UNIQUE: not present in any compared solution. Users verify access history without trusting the server
Server Knowledge	Server stores only encrypted bytes, wrapped keys, and IVs — same zero-knowledge property as all compared solutions but with formal threat model documentation

SecureDrive's primary novel contribution is the integration of a blockchain-anchored tamper-evident audit trail into a privacy-preserving cloud storage system. This is architecturally absent from all three examined solutions and represents a meaningful advance in the verifiability of access control in encrypted storage systems.

The secondary contribution is academic transparency: SecureDrive is the only system in this comparison to document a formal threat model (STRIDE), map its attack surface to the MITRE ATT&CK framework, and develop following the NIST SP 800-218 Secure Software Development Framework. This positions SecureDrive not merely as an implementation but as a formally analysed research prototype.

8. References

-
- Cryptomator Security Architecture. <https://docs.cryptomator.org/en/latest/security/architecture/>
 - Proton Drive Security Model. <https://proton.me/blog/protondrive-security> (Updated September 2022)
 - Filen Security and Privacy. <https://filen.io/security>
 - NIST SP 800-132: Recommendation for Password-Based Key Derivation, Part 1: Storage Applications. NIST, 2023.
 - NIST SP 800-218: Secure Software Development Framework (SSDF) Version 1.1. NIST, February 2022.
 - MITRE ATT&CK Framework for Enterprise. <https://attack.mitre.org/>
 - RFC 3394: Advanced Encryption Standard (AES) Key Wrap Algorithm. IETF, 2002.
 - Bellare M., Namprempe C.: Authenticated Encryption: Relations among notions and analysis of the generic composition paradigm. ASIACRYPT 2000.
-